Wrapper Classes in Java

Wrapper classes in Java are used to convert primitive data types into objects. They are part of the `java.lang` package and are essential for working with collection frameworks and scenarios where objects are required instead of primitives.

Autoboxing and Unboxing

Autoboxing

Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper class object. This feature simplifies code and enhances readability, as Java handles the conversion internally.

Example of Autoboxing:

```
public class AutoboxingExample {
    public static void main(String[] args) {
        int num = 10;
        Integer obj = num; // Autoboxing: primitive to wrapper class
        System.out.println("Wrapper Object: " + obj);
    }
}
```

Unboxing

Unboxing is the reverse process of autoboxing, where a wrapper class object is converted back into its corresponding primitive data type.

Example of Unboxing:

```
public class UnboxingExample {
    public static void main(String[] args) {
        Integer obj = 20;
        int num = obj; // Unboxing: wrapper class to primitive
        System.out.println("Primitive Value: " + num);
    }
}
```

Wrapper Classes in Java

Java provides the following wrapper classes for each primitive data type:

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Primitive Type	Wrapper Class
char	Character
boolean	Boolean

Collection Framework and Wrapper Classes

The Java Collection Framework (e.g., `ArrayList`, `HashMap`) does not allow primitive data types directly. Instead, it requires objects. Wrapper classes are used to bridge this gap by converting primitives into objects.

Example with Collection Framework:

```
import java.util.ArrayList;

public class CollectionExample {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>(); // Wrapper class
Integer

        // Autoboxing: Adding primitives as objects
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        // Unboxing: Retrieving values as primitives
        int firstNumber = numbers.get(0);
        System.out.println("First Number: " + firstNumber);

        // Printing all numbers
        System.out.println("All Numbers: " + numbers);
    }
}
```

Programming Examples

Example 1: Sum of Integers Using Wrapper Classes

```
public class WrapperSumExample {
    public static void main(String[] args) {
```

```
        Integer num1 = 15; // Autoboxing
        Integer num2 = 25; // Autoboxing

        int sum = num1 + num2; // Unboxing during addition
        System.out.println("Sum: " + sum);
    }
}
```

Example 2: Character Wrapper Class

```
public class CharacterExample {
    public static void main(String[] args) {
        Character ch = 'A'; // Autoboxing

        // Methods of Character class
        if (Character.isUpperCase(ch)) {
            System.out.println(ch + " is an uppercase letter.");
        }

        char lowerCh = Character.toLowerCase(ch); // Unboxing
        System.out.println("Lowercase: " + lowerCh);
    }
}
```

Example 3: Boolean Wrapper Class

```
public class BooleanExample {
    public static void main(String[] args) {
        Boolean flag = true; // Autoboxing

        if (flag) { // Unboxing in condition
            System.out.println("Flag is true!");
        }
    }
}
```

Interview Questions

1. **What are wrapper classes in Java, and why are they needed?**
   Wrapper classes are used to convert primitive data types into objects. They are required when working with collection frameworks or APIs that only accept objects.
2. **Explain autoboxing and unboxing in Java. Provide examples.**
   - Autoboxing: Automatic conversion of primitive to wrapper class.
   - Unboxing: Automatic conversion of wrapper class to primitive.
3. **Why does the Java Collection Framework not support primitive data types?**
   Collection frameworks operate on objects, not primitives, to leverage features like polymorphism and generics.
4. **What are some common methods provided by wrapper classes?**
   - `parseInt()` (Integer)
   - `toLowerCase()` (Character)
   - `valueOf()` (All wrapper classes)
5. **What is the difference between `valueOf()` and `parseInt()` in the `Integer` class?**
   - `valueOf()` returns an `Integer` object.
   - `parseInt()` returns a primitive `int` value.
6. **Can we use == to compare wrapper class objects?**

- For objects, == checks reference equality. To compare values, use `.equals()`.

7. **What is the default value of a wrapper class if declared as a class variable?**
   - Wrapper class objects default to `null` if not initialized.

By understanding wrapper classes, their usage, and integration with Java's features, you can write more flexible and efficient code. Use this document to solidify your knowledge and prepare for coding interviews!